

Image Restoration and Style-Conditioned Sketch Generation: A Progressive Deep Learning Pipeline

Aaiz Ur Rehman Kazmi
FAST-NUCES

GitHub: https://github.com/AaizUrRehmanKazmi/GenAI_A1

Models: https://github.com/AaizUrRehmanKazmi/GenAI_A1

Demo: <https://youtu.be/6zGzUnhm34M>

Abstract—This work addresses image restoration under unknown corruption and style-conditioned sketch generation through a progressive four-task deep learning pipeline. We develop and compare three restoration approaches—a universal convolutional autoencoder (Task 1), a hard-routed specialist ensemble with CNN-based corruption classification (Task 2), and a differentiable soft Mixture-of-Experts combining all branches through learnable routing weights (Task 3)—alongside a conditional GAN for multi-style facial photo-to-sketch translation (Task 4). All models are trained on the Oxford-IIIT Pet dataset (128×128 , RGB) with three synthetic corruption types (salt-and-pepper noise, Gaussian blur, rectangular occlusion) at multiple severities. The FS2K dataset provides paired photo-sketch supervision across three artist styles. We employ Optuna TPE search for hyperparameter optimization across all tasks, export final models to ONNX for deployment, and deliver a Docker-containerized web application with FastAPI backend and React frontend. Experimental results demonstrate that specialist routing substantially outperforms the universal baseline, while the soft MoE further improves restoration quality through differentiable expert blending.

I. INTRODUCTION

Image restoration aims to recover clean images from corrupted observations. Real-world corruptions are diverse and often unknown at inference time, making single-model approaches inherently limited. This work investigates a progressive complexity ladder:

- 1) **Universal Restoration** (Task 1): A single autoencoder handling all corruption types without knowing which corruption is present.
- 2) **Hard-Routed Specialists** (Task 2): A modular system that classifies the corruption type via a CNN, then dispatches to a specialized expert autoencoder.
- 3) **Soft Mixture-of-Experts** (Task 3): A differentiable extension where all experts contribute continuously weighted outputs via a learnable gate.
- 4) **Style-Conditioned Sketch Generation** (Task 4): A conditional GAN translating facial photographs into artist-style sketches using learned style embeddings.

The contributions of this work include: (1) a progressive architecture comparison from generalist to specialist to mixture approaches with quantitative routing analysis; (2)

a differentiable soft routing mechanism with temperature-scaled gating and load-balancing regularization; (3) a style-conditioned cGAN with learned embeddings in both generator and discriminator; and (4) a complete deployment pipeline with ONNX export, numerical parity verification, and a Dockerized web application.

II. RELATED RESEARCH

Autoencoders for restoration. Convolutional autoencoders compress images through an encoder bottleneck and reconstruct via a decoder. Dense (vector) bottlenecks lose spatial structure, while spatial bottlenecks preserve local information [1]. Our experiments confirm that transitioning from a 256-dimensional vector latent to a $32 \times 16 \times 16$ spatial latent significantly improves high-frequency reconstruction.

Mixture-of-Experts. Sparse and soft MoE approaches route inputs to specialized sub-networks [2]. Hard routing (argmax selection) is non-differentiable and prevents end-to-end training of the gate. Soft routing via softmax-weighted combination enables gradient flow through all experts and the gate simultaneously.

Conditional GANs. Pix2Pix [3] established paired image-to-image translation using U-Net generators with skip connections and PatchGAN discriminators. We extend this with learned style embeddings in both networks.

Hyperparameter optimization. Tree-structured Parzen Estimator (TPE) via Optuna [4] provides efficient Bayesian hyperparameter search with atomic persistence for resumable sessions.

III. DATASET PREPARATION

A. Oxford-IIIT Pet Dataset

We use an 80/20 train/validation split (seed 42) of the official trainval set: 2,944 training and 736 validation images. The 3,669 official test images are held out until final evaluation. All images are EXIF-transposed, bilinear-resized to 128×128 , and normalized to float32 $[0, 1]$.

TABLE I: Corruption types and severity parameters.

Type	Method	Severity Params
Salt-and-Pepper	Random pixel $\rightarrow$ 0/1	Density: 0.02, 0.06, 0.15
Gaussian Blur	2D Gaussian kernel	σ : 1.0, 2.0, 3.5
Occlusion	Rectangular zero patches	Area: 5–10%, 15–25%, 30–50%

B. Corruption Pipeline

Three corruption types are applied at three severity levels (Table I).

Training uses *dynamic* corruption (fresh randomization each access). Evaluation uses deterministic JSON manifests with 10 fixed cases per image (1 clean + 3×3 corruption/severity combinations), yielding 7,360 validation and 36,690 test specifications.

C. FS2K Dataset (Task 4)

The FS2K dataset [5] provides paired facial photographs and sketches across three artist styles. We use an 85/15 style-stratified validation split (seed 42): 898 training and 160 validation pairs, with synchronized horizontal flip augmentation.

IV. TASK 1: UNIVERSAL AUTOENCODER

A. Architecture

The `UniversalAutoencoder` uses a four-stage convolutional encoder with channels [16, 32, 64, 128], stride-2 downsampling (3 stages), ReLU activations, and Dropout(0.1). The baseline uses a 256-dimensional dense bottleneck (~ 4.4 M parameters).

B. Loss Function

$$\mathcal{L} = \alpha \cdot L_1(x, \hat{x}) + (1 - \alpha) \cdot (1 - \text{SSIM}(x, \hat{x})) \quad (1)$$

with $\alpha = 0.8$.

C. Bottleneck Investigation

The dense vector bottleneck causes severe spatial smoothing. Two spatial variants were investigated:

- **Spatial8AE**: $32 \times 8 \times 8$ latent (203K params)
- **Spatial16AE**: $32 \times 16 \times 16$ latent (203K params)

The 16×16 spatial latent retained significantly more high-frequency detail while maintaining the same parameter count.

D. Hyperparameter Search

Optuna TPE search over: learning rate $[2 \times 10^{-4}, 2 \times 10^{-3}]$ (log-uniform), batch size {8, 16}, latent dimension {128, 256, 512}, channel preset {standard, wide}, and $\alpha \in \{0.5, 0.65, 0.8\}$. The ranking objective is α -independent: $0.8 \cdot L_1 + 0.2 \cdot (1 - \text{SSIM})$.

E. Results

The baseline converges at epoch 13 out of 20. ONNX export achieves maximum parity error of 3.28×10^{-7} .

V. TASK 2: HARD-ROUTED SPECIALISTS

A. Architecture

Corruption Classifier: A CNN with BatchNorm, global average pooling, and 4-class output (clean, salt, blur, occlusion), trained with exactly balanced batches.

Specialist Autoencoders: Three independent Spatial16AE variants, each trained exclusively on its corruption type. Clean images bypass all models (exact identity).

Hard Router: At inference, $\hat{y} = \arg \max(\text{Classifier}(x))$ dispatches to the corresponding specialist.

B. Routing Analysis

We compare Oracle routing (ground-truth labels) versus Predicted routing:

$$\text{SSIM Cost} = \text{SSIM}_{\text{oracle}} - \text{SSIM}_{\text{predicted}} \quad (2)$$

$$\text{L1 Cost} = \text{L1}_{\text{predicted}} - \text{L1}_{\text{oracle}} \quad (3)$$

Three failure modes are identified:

- 1) **Clean-as-Corrupted**: Expert degrades sharp features on clean input.
- 2) **Corrupted-as-Clean**: Zero restoration (bypass on corrupted image).
- 3) **Cross-Corruption**: Wrong specialist applied.

C. Results

TABLE II: Task 2 selected validation results.

Metric	Value
Classifier Accuracy	98.48%
Macro F1	97.45%
Reconstruction L1	0.0278
Reconstruction SSIM	0.8493
Fixed- α Loss	0.0524

ONNX export: four separate graphs (classifier + 3 specialists). Maximum classifier parity error: 1.22×10^{-4} ; specialist errors $< 2 \times 10^{-6}$.

VI. TASK 3: SOFT MIXTURE-OF-EXPERTS

A. Architecture

The `SoftMoE` uses a temperature-scaled softmax gate over classifier logits:

$$w_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \quad \tau = 1.5 \quad (4)$$

with output:

$$\hat{x} = \sum_{i=0}^3 w_i \cdot E_i(x) \quad (5)$$

where E_0 = identity, $E_{1,2,3}$ = salt, blur, occlusion specialists.

B. Four-Term Loss

$$\mathcal{L} = 0.8L_1 + 0.2(1 - \text{SSIM}) + 0.1\mathcal{L}_{\text{CE}} + 0.01\mathcal{L}_{\text{balance}} \quad (6)$$

where $\mathcal{L}_{\text{CE}}$ is cross-entropy on gate logits and $\mathcal{L}_{\text{balance}} = \sum_i (\bar{w}_i - 1/4)^2$ prevents gate collapse.

C. Two-Stage Training

- 1) **Warm-up** (2 epochs): Experts frozen in eval mode, only gate trains, $\text{lr}=10^{-4}$.
- 2) **Joint** (8 epochs): All parameters trainable, $\text{lr}=2 \times 10^{-5}$.

Initialization: gate from Task 2 classifier (trial 005), experts from Task 2 specialists (trial 007).

D. Results

Selected trial 005, epoch 10: reconstruction $L_1 = 0.0239$, SSIM = 0.8734, fixed- α loss = 0.0444. ONNX parity: max error 1.85×10^{-6} .

VII. TASK 4: STYLE-CONDITIONED cGAN

A. Architecture

Generator (CandidateGenerator): Deep U-Net with 6 encoder levels, BatchNorm, skip connections, learned style embedding (3×8) concatenated to input channels, transposed convolution decoding, $\tanh \rightarrow [0, 1]$ output.

Discriminator (StylePatchGAN): Four convolutional stages producing 14×14 patch logits (70×70 receptive field) with separate learned style embedding.

B. Loss

$$\mathcal{L}_G = \mathcal{L}_{\text{BCE}}(\hat{D}, 1) + \lambda_{\text{rec}} L_1 + \lambda_{\text{edge}} \mathcal{L}_{\text{grad}} \quad (7)$$

$$\mathcal{L}_D = \mathcal{L}_{\text{BCE}}(D_{\text{real}}, 1) + \mathcal{L}_{\text{BCE}}(D_{\text{fake}}, 0) \quad (8)$$

with $\lambda_{\text{rec}} = 100$.

C. BatchNorm Diagnostic

A discriminator train/eval mode mismatch was discovered during the generator backward pass: running statistics were incorrectly updated during the generator-phase fake sample evaluation. The fix sets the discriminator to eval mode during the generator phase, preserving BatchNorm statistics from real-sample updates only.

D. Results

ONNX: Generator-only graph; inputs: image $[N, 3, 128, 128]$ + style $[N]$ (int64 0–2).

VIII. OPTIMIZATION AND EXPERIMENTAL SETUP

All tasks use Optuna TPE sampling with atomic JSON study persistence for safe checkpoint-resume across Colab/Kaggle sessions. Table III summarizes the search configuration.

TABLE III: Optuna search configuration per task.

Task	Parameters	Objective
1	lr , batch, latent, channels, α	$0.8L_1 + 0.2(1 - \text{SSIM})$
2 Clf	lr , batch, channels	CE loss
2 Spec	lr , batch, latent_ch, ch_preset, α	$0.8L_1 + 0.2(1 - \text{SSIM})$
3	lr_w , lr_j , τ , batch, α	$0.8L_1 + 0.2(1 - \text{SSIM})$
4	lr_G , lr_D , λ_{rec} , λ_{edge} , dropout	L_1 val loss

Experiment tracking uses MLflow with local file-based storage. All metrics, preview grids, and checkpoints are logged per epoch.

IX. APPLICATION AND DEPLOYMENT

The application uses a three-tier architecture: a React (Vite + Tailwind 4) frontend served via Nginx, proxying API calls to a FastAPI backend running ONNX Runtime inference. Docker Compose orchestrates both services with health-check probes.

A. API Endpoints

Four inference endpoints:

- POST /universal-restoration: Task 1 universal AE
- POST /hard-routing: Task 2 classifier + specialist routing
- POST /soft-mixture: Task 3 soft MoE with routing weights
- POST /face-to-sketch: Task 4 style-conditioned sketch

B. Frontend Workspaces

Four interactive workspaces provide: image upload with drag-and-drop, webcam capture (Task 4), classifier probability bars (Task 2), continuous routing weight visualization (Task 3), style selection cards (Task 4), side-by-side input/output comparison, inference timing, and result download.

C. ONNX Parity

TABLE IV: ONNX numerical parity verification.

Model	Max Abs Error
Task 1 Universal AE	3.28×10^{-7}
Task 2 Classifier	1.22×10^{-4}
Task 2 Salt Expert	1.58×10^{-6}
Task 2 Blur Expert	1.85×10^{-6}
Task 2 Occlusion Expert	1.67×10^{-6}
Task 3 Soft MoE	1.85×10^{-6}
Task 4 cGAN Generator	verified

X. LIMITATIONS AND CONCLUSION

A. Limitations

All models operate at 128×128 resolution, limiting fine detail recovery. Only three corruption types are addressed; real-world degradations remain uncovered. The dense bottleneck in Task 1 causes spatial smoothing despite spatial variant improvements. Task 4 GAN outputs exhibit blurred fine linework. Optuna searches use limited trial budgets due to GPU time constraints.

B. Conclusion

Progressively increasing architectural specialization yields consistent restoration improvements: hard routing via specialist experts outperforms the universal baseline, while soft MoE further improves through differentiable blending. Style-conditioned GANs successfully produce multi-style sketches from a single model. The complete pipeline demonstrates end-to-end machine learning engineering from data preparation through ONNX-deployed web application.

APPENDIX A AI USE

OpenAI Codex assisted with repository scaffolding, Docker configuration, infrastructure tests, data preparation scripts, model architecture stubs, training loop boilerplate, evaluation pipelines, ONNX export scripts, and the frontend placeholder shell. All trained model weights, research decisions, hyperparameter search interpretation, and final analysis are the student’s responsibility. The full chronological record is maintained in `docs/ai_use.md`.

REFERENCES

- [1] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, “Cats and dogs,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3498–3505.
- [2] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” *International Conference on Learning Representations (ICLR)*, 2017.
- [3] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.
- [4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.
- [5] D.-P. Fan *et al.*, “Facial-sketch synthesis: A new challenge,” in *Machine Intelligence Research*, 2022.